

Atividade: Gerenciando Permissões com chmod no Linux

Objetivos de Aprendizagem

- Compreender o modelo de permissões de arquivos no Linux (leitura, escrita, execução para dono, grupo e outros).
- Utilizar o comando chmod nos modos **simbólico** e **octal**.
- Ajustar permissões de arquivos e diretórios de forma segura.
- Aplicar permissões recursivamente com a opção -R.
- Resolver situações práticas do dia a dia de um administrador de sistemas.

Pré-requisitos

- Conhecimento básico de terminal Linux (navegação com cd, ls, pwd).
- Saber interpretar a saída de ls -l (ex.: -rw-r--r--).
- Um ambiente Linux (máquina virtual, WSL, ou laboratório real) com acesso ao terminal.

Materiais

- Computador com Linux (ou Live USB/VM).
- Terminal aberto.
- Usuário comum (não root) – os alunos devem criar seus próprios arquivos.

Duração sugerida

- 50 a 80 minutos (parte expositiva + prática guiada + exercícios individuais).

Roteiro da Atividade

1. Preparação do ambiente (5 min)

Cada aluno deve criar um diretório de trabalho e alguns arquivos de teste:

```
mkdir ~/lab_chmod
cd ~/lab_chmod
touch arquivo.txt script.sh dados.log
mkdir publico privado
echo "echo Olá" > script.sh
echo "conteúdo secreto" > privado/secreto.txt
```

2. Verificando permissões iniciais (5 min)

Execute ls -l e identifique:

- Quais permissões estão definidas para o dono, grupo e outros.
- Diferença entre arquivo e diretório (o d no início da linha).

3. Usando chmod no modo simbólico (10 min)

Comando	Significado
chmod u+x script.sh	Adiciona execução para o dono
chmod g-w arquivo.txt	Remove escrita para o grupo

Comando	Significado
<code>chmod o=r dados.log</code>	Define apenas leitura para outros
<code>chmod a=rwx publico</code>	Dá todas permissões a todos (dono, grupo, outros)

Prática: Altere as permissões do `script.sh` para que o dono possa executar, o grupo possa ler e outros não tenham acesso (`rwX r-- ---`).

Resposta esperada: `chmod u=rwx,g=r,o= script.sh`

4. Usando chmod no modo octal (10 min)

Recapitular a tabela octal:

Número	Permissão
0	---
1	--X
2	-W-
3	-WX
4	r--
5	r-X
6	rw-
7	rwX

Exercício guiado:

- `chmod 644 arquivo.txt` → dono pode ler/escrever; grupo e outros só leitura.
- `chmod 700 privado` → só o dono tem acesso total; ninguém mais.

Peça aos alunos que transformem `script.sh` para permissão 755 e verifiquem com `ls -l`.

5. Permissões em diretórios (explicação rápida) (5 min)

- **Leitura (r):** listar o conteúdo (`ls`).
- **Escrita (w):** criar ou remover arquivos dentro do diretório.
- **Execução (x):** entrar no diretório (`cd`).

Teste: `chmod 300 privado` (apenas escrita+execução para dono). O que acontece ao tentar `ls privado`? (Sem leitura, não é possível listar, mas ainda pode-se entrar se souber o nome do arquivo).

6. Recursividade com -R (5 min)

Crie uma estrutura:

```
mkdir -p docs/guia docs/imagens
touch docs/guia/README docs/imagens/figura.png
```

Torne toda a árvore `docs` com permissões 750 (dono total, grupo leitura+execução, outros nada):

```
chmod -R 750 docs/
```

Verifique com `ls -l docs` e `ls -l docs/guia`.

7. Exercícios práticos individuais (20 min)

Entregue a seguinte lista. Os alunos devem executar e anotar os comandos usados.

1. **Criação de um script executável**
Crie um arquivo `backup.sh` com o conteúdo `echo "Backup concluído"`. Use `chmod` para que **apenas o dono** possa executá-lo.
2. **Diretório colaborativo**
Crie um diretório `projeto`. Defina permissões para que:
 - Dono: leitura, escrita e execução (`rwX`).
 - Grupo: leitura e execução (`r-X`), mas **não escrita**.
 - Outros: sem acesso (`---`).
3. **Protegendo um arquivo**
O arquivo `senhas.txt` não pode ser lido, escrito ou executado por ninguém além do dono. Qual comando?
4. **Removendo permissão de escrita de outros**
Em um único comando, remova a permissão de escrita para **outros** no arquivo `dados.log` (usando modo simbólico).
5. **Permissões padrão em um diretório**
Dê permissão total (`rwX`) para todos no diretório `publico`. Depois, use `chmod 1777 publico` e explique o que o `1` (sticky bit) muda.
6. **Recursivo com símbolos**
No diretório `docs` (criado antes), use modo simbólico para **remover** a permissão de escrita do grupo recursivamente.

8. Respostas esperadas (para o professor)

1. `chmod 700 backup.sh` ou `chmod u=rwx,g=,o= backup.sh`
2. `chmod 750 projeto` ou `chmod u=rwx,g=rx,o= projeto`
3. `chmod 600 senhas.txt` ou `chmod u=rw,g=,o= senhas.txt`
4. `chmod o-w dados.log`
5. `chmod 777 publico`; depois `chmod 1777 publico` – o sticky bit impede que usuários removam arquivos de outros dentro do diretório.
6. `chmod -R g-w docs`

9. Desafio extra (para quem terminar cedo)

Crie um arquivo `teste.txt` com permissão `000`. Tente ler, escrever ou executar. Depois, como dono, use `chmod` para recuperar a permissão de leitura (`r--`). O que você observa?

(Resposta: mesmo sem permissão, o dono pode mudar as permissões, pois é o proprietário do arquivo.)

10. Perguntas para reflexão (discussão final)

- Por que é perigoso dar permissão `777` para arquivos em um servidor?
- Qual a diferença prática entre `chmod -R 700 pasta` e `chmod 700 pasta` quando a pasta contém subpastas?
- Como você tornaria um diretório acessível para leitura por outros, mas sem que eles possam ver os nomes dos arquivos? (Dica: isso é possível? Explique.)